Example 1: Resource monitoring in AWS with natural language prompting

Scenario: A cloud engineer needs to check the CPU utilsation of an EC2 instance in AWS using a natural language interface integrated with AWS CloudWatch.

Initial prompt (Poor): "Tell me about my server."

Issue: This prompt is vague. The system may not know which server, metric, or region the user is referring to, leading to irrelevant or incomplete responses.

Refined prompt (Effective): "Show me the CPU utilisation for EC2 instance i-1234567890abcdef0 in us-east-1 for the last 24 hours."

Outcome: The specificity of the instance ID, region, metric (CPU utilisation), and time frame ensures the AI or tool retrieves the exact data from CloudWatch. This reduces back-and-forth communication and saves time during critical operations.

Example 2: Automating infrastructure deployment with Terraform and prompting

Scenario: A DevOps engineer uses an AI-assisted tool to generate Terraform code for deploying a Kubernetes cluster on Google Cloud Platform (GCP).

Initial prompt (Poor): "Help me with Kubernetes on GCP."

Issue: The prompt lacks details about the desired configuration, such as node count, region, or networking setup, leading to generic or incomplete code suggestions.

Refined prompt (Effective): "Generate Terraform code to deploy a Kubernetes cluster on GCP with 3 worker nodes, in the us-central1 region, using a private network with CIDR range 10.0.0.0/16."

Outcome: The AI tool provides a tailored Terraform script that matches the exact requirements, minimising manual edits and deployment errors. This is especially valuable in cloud operations where misconfigurations can lead to security risks or downtime.

Example 3: Troubleshooting with Azure support chatbot

Scenario: A cloud administrator encounters an issue with a virtual machine (VM) on Microsoft Azure and uses an AI-powered support chatbot for assistance.

Initial prompt (Poor): "My VM isn't working."

Issue: The prompt lacks critical details like error messages, VM name, or resource group, making it difficult for the chatbot to provide actionable advice.

Refined prompt (Effective): "My VM named 'Prod-Web-01' in resource group 'RG-Prod' on Azure is in a failed state with error code 'ProvisioningFailed'. Can you suggest troubleshooting steps?"

Outcome: The chatbot identifies the specific issue based on the error code and resource details, providing targeted steps such as checking for quota limits or reattempting provisioning. This accelerates resolution in a production environment where downtime is costly.

Example 4: Cost optimisation with AI-driven cloud tools

Scenario: A cloud architect uses an AI-based cost management tool to identify unused resources in a multi-cloud environment (AWS and GCP).

Initial prompt (Poor): "Check my cloud costs."

Issue: The prompt is too broad, and the tool may return generic cost summaries without actionable insights.

Refined prompt (Effective): "Identify unused EBS volumes in AWS across all regions and idle Compute Engine instances in GCP for the past 30 days, and suggest deletion or resizing options."

Outcome: The tool provides a detailed report of specific unused resources, along with recommendations for cost savings, enabling the architect to take immediate action. This is critical in cloud operations where cost optimisation is a continuous priority.

Cloud management through prompt engineering

Cloud management refers to the process of overseeing, administering, and optimising cloud computing resources, services, and infrastructure to ensure efficiency, security, scalability, and cost-effectiveness. This includes tasks such as resource provisioning, monitoring, scaling, troubleshooting, cost optimisation, and ensuring compliance with organisational policies.

Cloud management through prompt engineering refers to leveraging carefully crafted prompts to interact with AI-driven tools, cloud platform APIs, or chatbots to manage cloud environments more efficiently. This approach is becoming increasingly relevant as cloud providers integrate AI and natural language processing (NLP) into their management consoles, support systems, and automation frameworks. Prompt engineering enables cloud professionals to streamline operations, reduce manual effort, and enhance decision-making by extracting actionable insights or automating tasks through precise communication with these systems.

Prompt engineering is critical for cloud management for several reasons.

Complexity of cloud environments: Modern cloud environments are complex, involving multi-cloud setups, hybrid infrastructures, and numerous services. Prompt engineering helps simplify interactions with these systems by translating high-level goals into specific, actionable commands.

Rise of AI-driven tools: Cloud providers like AWS, Microsoft Azure, and Google Cloud are increasingly embedding AI assistants and chatbots into their platforms (e.g., AWS Q, Azure Bot Service). Effective prompts ensure these tools deliver relevant and accurate responses.

Automation and efficiency: Well-engineered prompts can trigger automated